

MODELLO DI CALCOLO PER I LINGUAGGI CFL

Abbiamo visto Linguaggi Regolari Associati ad espressioni regolari (Reg, DFA) ed a DFA, NFA
...!

Abbiamo visto i CFL associati alle grammatiche CF.

UNA REG o UNA grammatica è UN SISTEMA formale che serve a generare le stringhe di UN CERTO linguaggio.

UN automa è una macchina, cioè UN DISPOSITIVO, che serve a RICONOSCERE una specifica stringa del linguaggio, cioè A RICONOSCERE se una certa stringa IN INPUT del linguaggio, appartiene al linguaggio oppure NO.

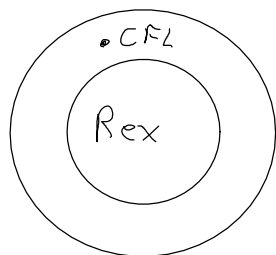
Le REG corrispondono ad automi a stati finiti e viceversa.

SE IO POSSO COSTRUIRE UNA REG PER UN linguaggio, POSSO COSTRUIRE UN automa a stati finiti PER quel linguaggio. E viceversa, quindi i due mondi sono equivalenti!

MA QUESTA COSA È VERA PER I CFL?

Gli automi a stati finiti sono UN modello di calcolo, cioè stiamo descrivendo UN modo di fare computazione e fare computazione è lo scopo principale.

ORA IL PUNTO DA CAPIRE È SE QUESTA grammatica di linguaggi contenuti free ABBIAMO O NO, UN modello di calcolo che deve ESSERE PIÙ POTENTE degli automi a stati finiti, perché sappiamo che i CFL sono UN SOTTOinsieme dei linguaggi Regolari.



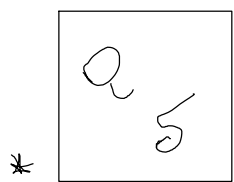
IL modello di calcolo che descrive la classe di linguaggi di CFL deve ESSERE STRETTAMENTE PIÙ POTENTE di quello degli automi a stati finiti.

CERTO CHE ESISTE QUESTO MODELLO DI CALCOLO!

↳ che descrive i linguaggi contenuti free (CFL)

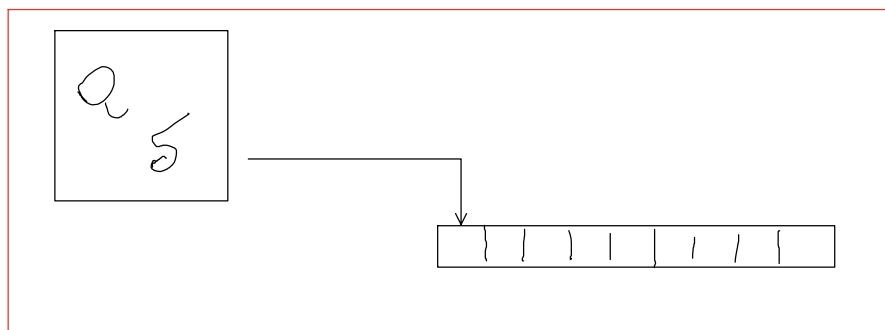
PDA (Push Down Automata)

UN automa ERA una macchina che AVEVA UN CONTROLLO: questo controllo poteva essere l'insieme degli stati e l'insieme delle transizioni che regolano i passaggi stato.



(quando leggo un certo INPUT in un determinato stato vado in un altro stato)

Quindi vedendolo come una macchina, questo controllo (*) ha una specie di TESTINA che legge dall'INPUT. L'INPUT lo potremmo vedere come una specie di NASTRO IN CUI OGNI CELLA CONTIENE UN SIMBOLO.



questo automa non fa altro che muovere la testina sul nastro; ogni volta che la testina si muove, legge il carattere successivo.

LA TESTINA PUÒ SOLTANTO SPOSTARSI PER LEGGERE UN CARATTERE SUCCESSIVO, NON PUÒ MAI TORNARE INDIETRO. NEL modello di calcolo degli automi a stati finiti NON C'È modo di TORNARE INDIETRO PER leggere un carattere già letto.

LA TESTINA SI MUOVE SEMPRE IN AVANTI.

Nei DFA AD OGNI PASSO DA TESTINA, SI MUOVE SEMPRE

NEGLI NFA, LA TESTINA PUÒ ANCHE STARE FERMA IN UN PASSO, SAREBBE LA FRECCIA ϵ , MA NON PUÒ TORNARE INDIETRO.

I PDA HANNO UN NASTRO DI INPUT E UNA TESTINA CHE SI MUOVE SOLAMENTE VERSO DESTRA, È TUTTO ANALOGO.

Qual'è la differenza fra un PDA e l'automa a stati finiti?

L'esistenza di una memoria infinita!

La memoria di un DFA è data soltanto dagli stati interni Q , e ogni macchina ha un insieme Q finito, per questo si chiama automa a stati finiti, la memoria della macchina è finita.

Significa che l'automa deve essere in grado di riconoscere linguaggi potenzialmente infiniti con una macchina finita, e questi linguaggi sono molto semplici, sono i linguaggi regolari.

IL PDA INVECE HA UNA MEMORIA INFINITA, MA QUESTA MEMORIA NON È UNA MEMORIA AD ACCESSO CASUALE, NON È UNA RAM, NON PUÒ ANDARE A PRENDERE UNA CELLA DI MEMORIA QUALUNQUE, NO, È UNA MEMORIA ORGANIZZATA CON UNO STACK.

UNA STRUTTURA LIFO (LAST IN FIRST OUT).

L'ultima cosa che metterò sulla mia struttura è la prima che posso leggere.

LO STACK DI UN PDA È INFINITO!

Questo stack viene gestito tramite operazioni di PUSH e POP.

Come si formalizza quindi un automa a stati finiti PDA?

È una tupla di entità che sono gli stati interni Q , l'alfabeto di input Σ (cioè i simboli che si trovano sul nastro di input).

Γ = "alfabeto dello stack" $\nearrow$ NASTRO

δ = "TRANSIZIONI"

q_0 = "STATO INIZIALE"

F = "STATI DI ACCETTAZIONE"

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

Questa macchina accetta la stringa, quando consuma tutta la stringa, ovvero infondo all'INPUT e la macchina si ritrova in uno stato di accettazione.

Se la macchina si trova in uno stato di accettazione e la stringa non è completamente consumata, la macchina non ha accettato quella stringa.

Questo è identico a quello che succede con gli automi a stati finiti.

Prendiamo una stringa w . $w \in \Sigma^*$, w appartenente alla stringa di input.

QUANDO $P(w)$ accetterà la stringa? $w \in L(P)$?

SE POSSIAMO SCRIVERE UNA COSA DEL GENERE:

$$w = w_1 w_2 w_3 w_4 \dots w_m, \quad w_i \in \Sigma \cup \{\epsilon\}.$$

ed esiste un $R_0, R_1, R_2, \dots, R_m$, dove $R_i \in Q$ (sono stati della macchina).

Cosa contiene lo stack in ogni stato?

deve esistere una sequenza $S_0, S_1, S_2, \dots, S_m$, dove ogni S_i non è altro che una stringa dell'alfabeto dello stack:

$$S_i \in \Gamma^*$$

all'inizio:

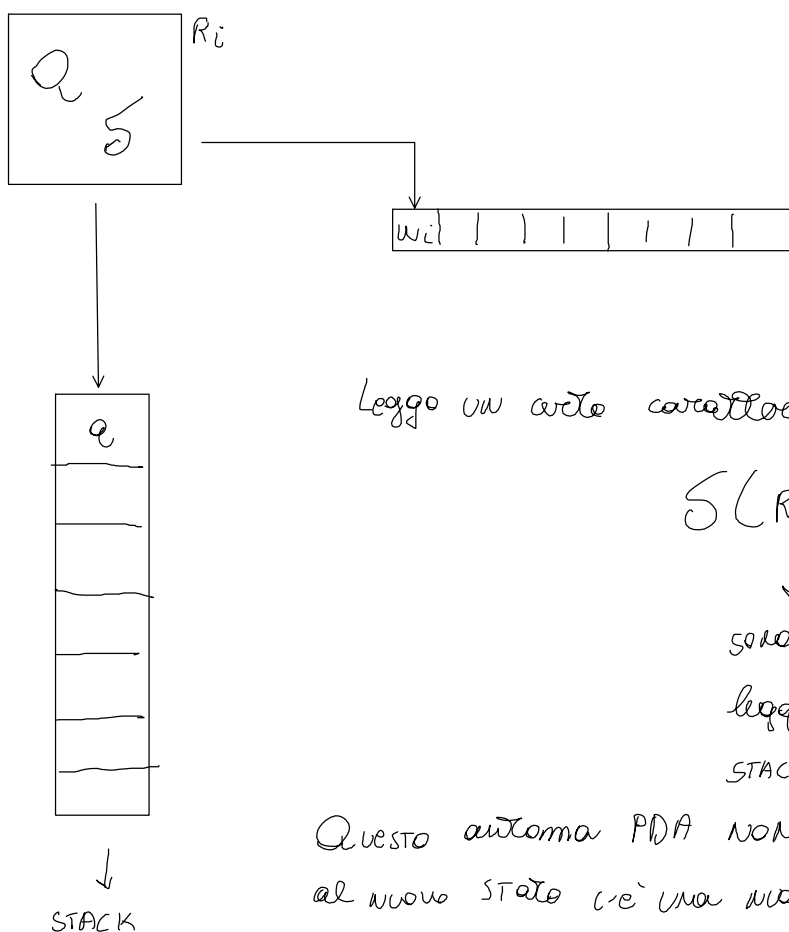
$$R_0 = q_0$$

$S_0 = \epsilon \rightarrow$ i caratteri dello stack sono vuoti, non c'è nulla.

quando consumo tutto l'input della stringa, allora accetto, perciò $R_m \in \Gamma$.

TRANSIZIONI

Ad ogni istante sto leggendo un carattere dell'input, facciamo conto il carattere iesimo!



Leggo un certo carattere w_i e sono in un certo stato R_i e memorizzo a .

$$\delta(R_i, w_{i+1}, a)$$



sono nello stato R_i , il successivo carattere da leggere w_{i+1} , e a è ciò che c'è sulla cima dello STACK.

Questo automa PDA NON è deterministico. Quindi non esiste che al nuovo stato c'è una nuova cima dello stack.

$$\delta: Q \times \Sigma_E \times \Gamma_E \rightarrow \mathcal{P}(Q \times \Gamma_E)$$

$$(q_i, w_i, a) \rightarrow \{(q', b'), (q'', b'')\}$$

↳ push → push il valore se $b \in \Gamma$

se $a \in \Gamma$ allora la transizione nella lettura è una POP.

se $a \in E$ allora non ci importa nulla.

se $b \in E$ stai facendo la transizione senza mettere nulla nello stack

se $b \in \Gamma$ push il valore ad ogni transizione.

Ogni volta che fai una transizione fai una push del valore.